

PROJECT DESIGN PHASE -II

SOLUTION REQUIREMENTS(FUNCTIONAL&NON FUNCTIONAL)

Date	02 October 2026
Team ID	SWTID-2026-1180
Project Name	Implement Client Script and UI Policy (incidents)
Maximum Marks	4 Marks

Functional Requirements • Dynamic Form

Orchestration (UI Policies):

- The system must automatically show, hide, or make fields mandatory based on user-selected criteria (e.g., Category, Subcategory, or Priority) without requiring page reloads.
- The form must default to a clean, minimal view by hiding advanced or irrelevant technical fields on initial load.
- **Form Initialization & User Context (onLoad):**
 - The form must automatically query session data (`g_user`) upon loading to prepopulate caller details (such as name, email, and department) and reduce manual data entry.
- **Real-Time Input Validation (onChange):**
 - Fields requiring specific formatting (such as phone numbers, dates, or email formats) must be evaluated instantly in the browser using regular expressions as the user types.
 - The form must display clear inline error or helper messages (`g_form.showFieldMsg`) immediately beneath the field when validation fails, without disrupting the user's workflow.
- **Asynchronous Data Fetching (GlideAjax):**
 - The system must query backend records (such as Configuration Item/Asset status or warranty contracts) asynchronously via Script Includes without freezing the browser user interface.
- **Pre-Submission Gatekeeping (onSubmit):**
 - The form must execute a final validation check when the user clicks **Submit**. If any required data is missing or malformed, submission must be aborted (`return false`) directly in the browser while preserving all entered text and preventing full-page reloads.

Non-Functional Requirements

- **Performance & Speed:**
 - Form load times and client script execution must complete in under 2 seconds to eliminate latency bottlenecks.

- Asynchronous calls (`GlideAjax`) must execute in the background to ensure zero UI freezing.
- **Data Integrity & Quality:**
 - The solution must reduce form completion error rates, eliminate corrupted data entry, and prevent unnecessary server-side transaction rejections.
 - Enforce strict role-based or state-based field locking (e.g., setting records to read-only when resolved) to protect historical audit data.
- **Maintainability & Usability:**
 - Leverage declarative, no-code UI Policies wherever possible for visibility rules to minimize custom JavaScript maintenance overhead for administrators.
 - Ensure mobile compatibility so that client-side scripts and UI policies execute smoothly across standard web browsers and ServiceNow mobile interfaces.